

PROYECTO “FOTAZA 2” PROGRAMACIÓN WEB II

Objetivos del TPI

Este trabajo integrador tiene como objetivo principal que los estudiantes puedan desarrollar y poner en práctica los conocimientos adquiridos en la asignatura a partir del desarrollo de un caso de estudio.

Especificación del problema

Se necesita implementar una aplicación web que permita almacenar, ordenar, buscar, vender y compartir fotografías en línea, a través de Internet.

La aplicación debe propiciar la generación de una comunidad de usuarios que puedan compartir fotografías creadas por ellos mismos. La comunidad se debe regir por normas de comportamiento y condiciones de uso que favorezcan la buena gestión de los contenidos.

Si bien las funcionalidades de la aplicación no estarán limitadas, a continuación, se listan las principales funcionalidades que deberá contener la aplicación.

Funcionalidades (Requerimientos mínimos):

1.- Sistema de autenticación de usuarios.

- Solo usuarios registrados en la aplicación pueden interactuar con la app (subir contenido, comentar, votar, denunciar, etc)
- Usuarios anónimos solo pueden ver contenido público (imágenes sin Copyright).

2.- Gestor de contenidos (imágenes)

- Los usuarios pueden postear una publicación la cual tiene un título, descripción (opcional), 1 o más imágenes y 1 o más etiquetas.
- Las imágenes de las publicaciones pueden ser denunciadas por los usuarios. Para denunciarlas el usuario debe elegir un motivo de denuncia y hacer una descripción o justificación de la denuncia. Una vez que la imagen ha recibido una denuncia, la publicación que la contiene no puede ser modificada por el usuario. Cuando una imagen tiene más de 3 denuncias de diferentes usuarios, el sistema coloca la publicación en la lista de trabajo del validador de contenidos (perfil de la app). El validador de contenidos puede dar de baja la publicación o desestimar las denuncias. Cuando un usuario llega a 3 publicaciones “bajadas” el sistema inactiva su cuenta.
- Las imágenes de las publicaciones pueden ser comentadas por cualquier usuario. En cualquier momento el autor de la publicación puede cerrar los comentarios para la imagen. La imagen mostrará los comentarios registrados hasta el cierre.
- Los comentarios (que no son del autor) también pueden ser denunciados. En estos casos el autor

puede ver de forma organizada las denuncias y si llega a corresponder podrá borrar el comentario.

- Cada imagen publicada puede ser valorizada por los usuarios una sola vez. No puede valorizarla el autor. Siempre debe mostrar la valorización promedio y la cantidad de valoraciones.
- Imágenes que estén bien valorizadas y con una cantidad de votos “considerable” deben tener preferencia para mostrarse en la home de la aplicación. Debe considerarse también la posibilidad de que puedan formar parte de la home imágenes que no estén en ese grupo para que haya un balance. Uds deben definir el criterio para esta regla.
- Cuando se crea una publicación cada imagen debe tener o definir una licencia. Estas pueden ser con copyright o sin copyright. En caso de estar protegidas con copyright la aplicación debe permitir proteger la imagen con una marca de agua (la cual puede tener un texto personalizado por el autor).
- Los usuarios pueden notificar al autor por medio de un botón “me interesa” el interés de adquirir la imagen. En estos casos el sistema comparte el perfil del interesado con el autor y pone a disposición una mensajería privada para que ambos se puedan contactar y llegar a un acuerdo.

3.- Motor de búsqueda de contenidos.

La app debe permitir buscar/recuperar las imágenes/publicaciones a partir de un motor de búsqueda potente y flexible. Se requiere implementar diferentes tipos de filtros/parámetros incluso que puedan ser combinados entre sí.

4.- Seguimiento de usuario (Followers)

Los usuarios podrán seguir a otros usuarios dentro de la plataforma para ver su contenido más fácilmente.

- Un usuario puede seguir o dejar de seguir a otro usuario.
- Cada perfil debe mostrar:
 - Cantidad de seguidores.
 - Cantidad de usuarios seguidos.
- La aplicación debe contar con una sección “Publicaciones de usuarios que sigo”.
- Un usuario no puede seguirse a sí mismo.
- El sistema debe evitar seguir al mismo usuario más de una vez.

5.- Gestor de notificaciones

El sistema debe notificar a los usuarios cuando ocurre alguna interacción relevante con su contenido.

Eventos que generan notificaciones:

- Cuando alguien comenta una publicación.
- Cuando alguien valoriza una imagen.

- Cuando alguien marca “me interesa” en una imagen.
- Cuando alguien comienza a seguir al usuario.

Requerimientos:

- Cada usuario debe tener una sección “Notificaciones”.
- Las notificaciones deben indicar:
 - tipo de evento
 - usuario que generó la acción
 - fecha
- Debe poder marcarse una notificación como leída.

6.- Gestión de colecciones o favoritos

Los usuarios podrán guardar publicaciones o imágenes en colecciones personales.

Requerimientos:

- Un usuario puede guardar una publicación como favorita.
- Los favoritos solo pueden ser vistos por el usuario.
- Un usuario puede crear colecciones (ej: “Paisajes”, “Ideas”, “Inspiración”).
- Una colección puede contener varias publicaciones.
- No se puede guardar la misma publicación dos veces en la misma colección.

Pautas de entrega y presentación

El alumno deberá entregar la aplicación subida a un repositorio github. Este repositorio deberá mantener el histórico tanto de las sesiones de trabajo diarias como así también las nuevas funcionalidades desarrolladas.

Importante!! No se aceptarán entregas cuyos repositorios no refleje el trabajo desarrollado en el proyecto. Por ejemplo, repositorios con pocos commits, commits excesivamente grandes, commits sin cambios significativos, etc

La aplicación deberá ser subida y presentada desde un servidor real en internet. Considere este aspecto desde el comienzo del proyecto. Considere los servidores disponibles para Node y la disponibilidad o no de servicios de gestión de bases de datos mysql o postgresQL.

Junto con las fuentes del proyecto se debe disponer de un readme que disponga de la información necesaria para desplegar el proyecto en un entorno local como así también un breve informe que cuente con que problemas se encontró y como los soluciono durante el desarrollo del proyecto.

Respecto de la BD, el proyecto debe tener en su raíz una copia de seguridad (sql) que permita la creación/restoración de la BD. La copia de seguridad debe tener los datos necesarios para poder hacer

una prueba del sistema entregado (usuarios, registros de tablas auxiliares, registros de configuración, información de ejemplo, etc). De mas esta decir que deben existir tantos usuarios de pruebas como roles existan. La información de estos usuarios debe estar en el readme.

La BD debe estar correctamente configurada y puede ser evaluada durante la presentación del proyecto. Debe ser clara, robusta y bien documentada.

Se evaluará que:

- Las entidades y relaciones estén bien definidas.
- Que este normalizada mínimo 3FN
- Uso adecuado de Claves primarias e índices.
- Correcta integridad referencial
- Uso adecuado de tipos de datos.
- Implementación de restricciones (null, default, check, etc)

Respecto del desarrollo de la aplicación esta debe realizarse usando Node y Express como plataforma principal. La aplicación debe renderizar las vistas totalmente en el lado del servidor utilizando PUG. La aplicación podrá utilizar cualquier librería que sea necesaria en el lado del servidor y para el front-end podrá reutilizar plantillas CSS, frameworks de CSS (Bootstrap, tailwind, Materialize u otros).

Importante!! No se aceptarán proyectos que utilicen desarrollos con frameworks del front como react, vue, angular o frameworks del back como Next.js, Nuxt.js, gatsby, etc.

Cualquier información necesaria para poder evaluar el trabajo presentado debe estar escrita en el readme del proyecto.

Condiciones para aprobar el proyecto

- El proyecto se defiende en mesa de examen para **Aprobar** la materia. La materia no es promocionable.
- Se valorará especialmente la creatividad y la correcta aplicación de buenas prácticas.
- **El cumplimiento de los requisitos funcionales es indispensable para aprobar el trabajo.**
- El código debe estar disponible en un repositorio remoto (GitHub o similar) para su revisión.
- La aplicación debe correr en un servidor accesible por internet.
- La documentación debe permitir la instalación y prueba del sistema sin necesidad de intervención del desarrollador.

Recomendaciones

- Realizar tempranamente un análisis del dominio de la aplicación y definir el **alcance** del proyecto.
- Evacuar todas las dudas respecto a los requerimientos funcionales en el comienzo del proyecto.
- Organizar de forma efectiva los archivos fuentes de la aplicación como así también la claridad del código escrito.

Criterios de evaluación del proyecto

Criterio	Descripción	Excelente (9-10)	Bueno (7-8)	Regular (4-6)	Insuficiente (1-3)	Ponderación (%)
1. Diseño y Arquitectura	Estructura organizada, separación clara en modelos, rutas y controladores. Uso de buenas prácticas de programación.	Estructura modular, código limpio y reutilizable.	División clara, pero con algunas redundancias.	Código funcional pero desordenado, con poca reutilización.	Código desordenado, sin modularidad clara.	20%
2. Base de Datos (MySQL)	Diseño relacional, normalización, claves primarias y foráneas, integridad referencial.	Modelo relacional correcto, sin redundancias y con restricciones.	Diseño adecuado, pero con algunas redundancias menores.	Modelo básico, con errores de diseño o falta de normalización.	Modelo deficiente, sin relaciones claras ni restricciones.	10%
3. Autenticación y Seguridad	Login con JWT o sesiones, encriptación de contraseñas, validaciones y protección contra inyecciones SQL.	Autenticación robusta, con validaciones y cifrado adecuado.	Autenticación funcional, pero con algunas carencias de seguridad.	Autenticación básica, con validaciones mínimas.	No implementa autenticación o es insegura.	10%
4. Funcionalidad y Operatividad	Gestión de pacientes, alta, baja, modificación, consulta y validación de datos obligatorios.	Cumple con todos los requisitos y funciona correctamente.	Cumple con la mayoría de los requisitos, con errores menores.	Cumple parcialmente con los requisitos, con errores importantes.	No cumple con los requisitos o la aplicación no funciona.	25%
5. Interfaz de Usuario (Opcional)	Diseño, usabilidad y correcta integración con el backend.	Diseño atractivo, fácil de usar y bien integrado.	Diseño aceptable, con algunas deficiencias de usabilidad.	Diseño básico, con poca integración con el backend.	No presenta interfaz o la integración es deficiente.	25%
6. Documentación	README con instrucciones, documentación de endpoints y comentarios en el código.	Documentación completa, clara y bien estructurada.	Documentación adecuada, aunque con algunos faltantes.	Documentación básica y poco clara.	No presenta documentación o es insuficiente.	10%

Condiciones para regularizar la materia

- Se deberá entregar en la actividad del aula que se indique los siguientes artefactos:
 - Dirección en github donde se encuentre el proyecto.
 - Captura de github donde figure listado de commits del proyecto.
 - Dirección de la aplicación en producción para ser probada. (servidor en internet)
 - Video de no más de 3 minutos que muestre el uso de la aplicación. Debe mostrar como la app satisface los requisitos de regularización. La app del video debe estar subida al servidor de internet para la prueba.

- Se evaluará siguiendo el mismo criterio utilizado para la aprobación. Teniendo en cuenta que la aplicación debe cumplir todos los requisitos siguientes para regularizar:
 - Creación de publicación.
 - Buscador de publicaciones/imágenes
 - Módulo de comentarios.
 - Valoración de imágenes
 - Seguimiento de usuarios
- Toda aclaración, documentación e información sobre usuarios de prueba debe estar en el readme del proyecto.
- El proyecto debe utilizar variables de entorno para la configuración (base de datos, puertos, claves, etc). Debe incluirse un archivo: .env.example
- Una vez clonado el repositorio, la aplicación debe poder instalar todas sus dependencias ejecutando: npm install.
- El proyecto debe incluir un script SQL o mecanismo de inicialización para crear las tablas necesarias. npm run db:init
- La aplicación debe poder ejecutarse mediante el siguiente comando: npm start
- Una vez iniciado el servidor, la aplicación debe quedar accesible en: http://localhost:3000

Importante!! El proyecto **no será considerado correctamente entregado** si no puede ejecutarse siguiendo únicamente estos pasos:

1. Clonar el repositorio
2. Ejecutar npm install
3. Ejecutar npm run db:init
4. Configurar .env
5. Ejecutar npm start